Chapter 10
Introduction to Artificial Neural Networks with Keras

Outline

- Implementing MLPs with Keras

Exercise: Tensorflow Playground

Tensorflow Playground

https://playground.tensorflow.org/

- The TensorFlow Playground is a handy neural network simulator built by the TensorFlow team.
- In this exercise, you will train several binary classifiers in just a few clicks, and tweak the model's architecture and its hyperparameters to gain some intuition on how neural networks work and what their hyperparameters do.
- Take some time to explore the following:
  - **The patterns learned by a neural net.**
    - Try training the default neural network by clicking the Run button (top left). Notice how it quickly finds a good solution for the classification task.
    - The neurons in the first hidden layer have learned simple patterns, while the neurons in the second hidden layer have learned to combine the simple patterns of the first hidden layer into more complex patterns.
    - In general, the more layers there are, the more complex the patterns can be.

Tensorflow Playground

- **Activation functions.**
  - Try replacing the tanh activation function with a ReLU activation function, and train the network again.
  - Notice that it finds a solution even faster, but this time the boundaries are linear.
  - This is due to the shape of the ReLU function.
- **The risk of local minima.**
  - Modify the network architecture to have just one hidden layer with three neurons.
  - Train it multiple times (to reset the network weights, click the Reset button next to the Play button).
  - Notice that the training time varies a lot, and sometimes it even gets stuck in a local minimum.

Tensorflow Playground

- **What happens when neural nets are too small.**
  - Remove one neuron to keep just two.
  - Notice that the neural network is now incapable of finding a good solution, even if you try multiple times.
  - The model has too few parameters and systematically underfits the training set.
- **What happens when neural nets are large enough.**
  - Set the number of neurons to eight, and train the network several times.
  - Notice that it is now consistently fast and never gets stuck.
  - This highlights an important finding in neural network theory: large neural networks almost never get stuck in local minima, and even when they do these local optima are almost as good as the global optimum.
  - However, they can still get stuck on long plateaus for a long time.

Tensorflow Playground

- **The risk of vanishing gradients in deep networks**.
  - Select the spiral dataset (the bottom-right dataset under “DATA”), and change the network architecture to have four hidden layers with eight neurons each.
  - Notice that training takes much longer and often gets stuck on plateaus for long periods of time.
  - Also notice that the neurons in the highest layers (on the right) tend to evolve faster than the neurons in the lowest layers (on the left).
  - This problem, called the “vanishing gradients” problem, can be alleviated with better weight initialization and other techniques, better optimizers (such as AdaGrad or Adam), or Batch Normalization (discussed in Chapter 11).

Implementing MLPs with Keras

Keras

- Keras is TensorFlow's high-level deep learning API: it allows you to build, train, evaluate, and execute all sorts of neural networks.
- The original Keras library was developed by Francois Chollet as part of a research project and was released as a standalone open source project in March 2015.
- It quickly gained popularity, owing to its ease of use, flexibility, and beautiful design.
- Keras was officially chosen as its preferred high-level API when TensorFlow 2 came out.
- Installing TensorFlow will automatically install Keras as well, and Keras will not work without TensorFlow installed.

Keras

- Colab runtimes come with recent versions of TensorFlow and Keras preinstalled.
- However, if you want to install them on your own machine, please see the installation instructions at *https://homl.info/install*.

Building an Image Classifier Using the Sequential API

Building an Image Classifier Using the Sequential API

- First, will use the Fashion MNIST dataset:
  - It has the exact same format as MNIST: 70,000 grayscale images of 28x28 pixels each, with 10 classes,
  - The images represent fashion items rather than handwritten digits,
  - Each class is more diverse, and the problem turns out to be significantly more challenging than MNIST.
  - For example, a simple linear model reaches about 92% accuracy on MNIST, but only about 83% on Fashion MNIST.

Using Keras to load the dataset

- Keras provides some utility functions to fetch and load common datasets, including MNIST, Fashion MNIST, and a few more.
- It's already shuffled and split into a training set (60,000 images) and a test set (10,000 images), but we'll hold out the last 5,000 images from the training set for validation:

```
import tensorflow as tf

fashion_mnist = tf.keras.datasets.fashion_mnist.load_data()
(X_train_full, y_train_full), (X_test, y_test) = fashion_mnist
X_train, y_train = X_train_full[:-5000], y_train_full[:-5000]
X_valid, y_valid = X_train_full[-5000:], y_train_full[-5000:]
```

- TensorFlow is usually imported as `tf`, and the Keras API is available via `tf.keras`.

Using Keras to load the dataset

- When loading MNIST or Fashion MNIST using Keras rather than Scikit-Learn, one important difference is that every image is represented as a 28x 28 array rather than a 1D array of size 784.
- Moreover, the pixel intensities are represented as integers (from 0 to 255) rather than floats (from 0.0 to 255.0).
- Let's take a look at the shape and data type of the training set:

```
>>> X_train.shape
(55000, 28, 28)
>>> X_train.dtype
dtype('uint8')
```

Pre-processing data

- For simplicity, we'll scale the pixel intensities down to the 0–1 range by dividing them by 255.0 (this also converts them to floats):

```
X_train, X_valid, X_test = X_train / 255., X_valid / 255., X_test / 255.
```

- We need the list of class names:

```
class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
               "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]
```

For example, the first image in the training set represents an ankle boot:

```
>>> class_names[y_train[0]]
'Ankle boot'
```



Figure 10-10. Samples from Fashion MNIST

Creating the model using the sequential API

- Here is a classification MLP with two hidden layers:

```
tf.random.set_seed(42)
model = tf.keras.Sequential()
model.add(tf.keras.layers.Input(shape=[28, 28]))
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(300, activation="relu"))
model.add(tf.keras.layers.Dense(100, activation="relu"))
model.add(tf.keras.layers.Dense(10, activation="softmax"))
```

Code explanations – line by line

- **First line**: set TensorFlow's random seed to make the results reproducible: the random weights of the hidden layers and the output layer will be the same every time you run the notebook. You could also choose to use the `tf.keras.utils.set_random_seed()` function, which conveniently sets the random seeds for TensorFlow, Python (`random.seed()`), and NumPy (`np.random.seed()`).
- **Second line**: creates a `Sequential` model. This is the simplest kind of Keras model for neural networks that are just composed of a single stack of layers connected sequentially. This is called the sequential API.
- Next, we build the first layer (an `Input` layer) and add it to the model. We specify the input `shape`, which doesn't include the batch size, only the shape of the instances. Keras needs to know the shape of the inputs so it can determine the shape of the connection weight matrix of the first hidden layer.

Code explanations – line by line

- Then we add a `Flatten` layer. It converts each input image into a 1D array: for example, if it receives a batch of shape [32, 28, 28], it will reshape it to [32, 784]. In other words, if it receives input data `X`, it computes `X.reshape(-1, 784)`. This layer doesn't have any parameters; it's just there to do some simple preprocessing.
- Next we add a `Dense` hidden layer with 300 neurons. It will use the ReLU activation function. Each `Dense` layer manages its own weight matrix, containing all the connection weights between the neurons and their inputs. It also manages a vector of bias terms (one per neuron). When it receives some input data, it computes Equation 10-2.
- Then we add a second `Dense` hidden layer with 100 neurons, also using the ReLU activation function.
- Finally, we add a `Dense` output layer with 10 neurons (one per class), using the softmax activation function because the classes are exclusive.

Equation 10-2. Computing the outputs of a fully connected layer

$$h_{\mathbf{W},\mathbf{b}}(\mathbf{X}) = \phi(\mathbf{XW} + \mathbf{b})$$

Note

- Specifying `activation="relu"` is equivalent to specifying `activation=tf.keras.activations.relu`.
- Other activation functions are available in the `tf.keras.activations` package.
- See https://keras.io/api/layers/activations for the full list.

Building the model – second way

- Instead of adding the layers one by one as we just did, it's often more convenient to pass a list of layers when creating the `Sequential` model.
- You can also drop the `Input` layer and instead specify the `input_shape` in the first layer:

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dense(300, activation="relu"),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

Summarizing the model

- The model's `summary()` method displays all the model's layers, including each layer's name (which is automatically generated unless you set it when creating the layer), its output shape (`None` means the batch size can be anything), and its number of parameters.
- The summary ends with the total number of parameters, including trainable and non-trainable parameters.
- Here we only have trainable parameters (you will see some non-trainable parameters later in this chapter):

```
>>> model.summary()
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 flatten (Flatten)           (None, 784)               0

 dense (Dense)               (None, 300)               235500

 dense_1 (Dense)             (None, 100)               30100

 dense_2 (Dense)             (None, 10)                1010

=================================================================
Total params: 266,610
Trainable params: 266,610
Non-trainable params: 0
_________________________________________________________________
```

Summarizing the model

- Note that `Dense` layers often have a lot of parameters. For example, the first hidden layer has 784 x 300 connection weights, plus 300 bias terms, which adds up to 235,500 parameters!
- This gives the model quite a lot of flexibility to fit the training data, but it also means that the model runs the risk of overfitting, especially when you do not have a lot of training data.
- Each layer in a model must have a unique name (e.g., `"dense_2"`). You can set the layer names explicitly using the constructor's `name` argument, but generally **it's simpler to let Keras name the layers automatically**, as we just did.
- **Keras also ensures that the name is globally unique**, even across models, by appending an index if needed, as in `"dense_2"`. But why does it bother making the names unique across models? Well, this makes it possible to merge models easily without getting name conflicts.
  - All global state managed by Keras is stored in a Keras session, which you can clear using `tf.keras.backend.clear_session()`. In particular, this resets the name counters.

Layers of the model

- You can easily get a model's list of layers using the `layers` attribute, or use the `get_layer()` method to access a layer by name:

```
>>> model.layers
[<keras.layers.core.flatten.Flatten at 0x7fa1dea02250>,
 <keras.layers.core.dense.Dense at 0x7fa1c8f42520>,
 <keras.layers.core.dense.Dense at 0x7fa188be7ac0>,
 <keras.layers.core.dense.Dense at 0x7fa188be7fa0>]
>>> hidden1 = model.layers[1]
>>> hidden1.name
'dense'
>>> model.get_layer('dense') is hidden1
True
```

Parameters of Layers

- All the parameters of a layer can be accessed using its `get_weights()` and `set_weights()` methods. For a `Dense` layer, this includes both the connection weights and the bias terms:

```
>>> weights, biases = hidden1.get_weights()
>>> weights
array([[ 0.02448617, -0.00877795, -0.02189048, ...,  0.03859074, -0.06889391],
       [ 0.00476504, -0.03105379, -0.0586676 , ..., -0.02763776, -0.04165364],
       ...,
       [ 0.07061854, -0.06960931,  0.07038955, ..., 0.00034875,  0.02878492],
       [-0.06022581,  0.01577859, -0.02585464, ..., 0.00272203, -0.06793761]],
       dtype=float32)
>>> weights.shape
(784, 300)
>>> biases
array([0., 0., 0., 0., 0., 0., 0., 0., 0., ...,  0., 0., 0.], dtype=float32)
>>> biases.shape
(300,)
```

Initialization of weights

- Notice that the `Dense` layer initialized the connection weights randomly (which is needed to break symmetry, as discussed earlier), and the biases were initialized to zeros, which is fine.
- If you want to use a different initialization method, you can set `kernel_initializer` (kernel is another name for the matrix of connection weights) or `bias_initializer` when creating the layer.
- Full list of initializers is at *https://keras.io/api/layers/initializers*.

NOTE

- The shape of the weight matrix depends on the number of inputs, which is why we specified the `input_shape` when creating the model.
- If you do not specify the input shape, it's OK: Keras will simply wait until it knows the input shape before it actually builds the model parameters. This will happen either when you feed it some data (e.g., during training), or when you call its `build()` method.
- Until the model parameters are built, you will not be able to do certain things, such as display the model summary or save the model.
- So, if you know the input shape when creating the model, it is best to specify it.

Compiling the model

- After a model is created, you must call its `compile()` method to specify the **loss function** and the **optimizer** to use. Optionally, you can specify a list of extra metrics to compute during training and evaluation:

```
model.compile(loss="sparse_categorical_crossentropy",
              optimizer="sgd",
              metrics=["accuracy"])
```

- Using `loss="sparse_categorical_crossentropy"` is the equivalent of using `loss=tf.keras.losses.sparse_categorical_crossentropy`.
- Similarly, using `optimizer="sgd"` is the equivalent of using `optimizer=tf.keras.optimizers.SGD()`,
- Using `metrics=["accuracy"]` is the equivalent of using `metrics=[tf.keras.metrics.sparse_categorical_accuracy]` (when using this loss)
- Other losses, optimizers, and metrics exists; for the full lists, see https://keras.io/api/losses, https://keras.io/api/optimizers, and https://keras.io/api/metrics.

Explanation of loss function

- We use the `"sparse_categorical_crossentropy"` loss because we have **sparse labels** (i.e., for each instance, there is just a target class index, from 0 to 9 in this case), and the classes are exclusive.
- If instead we had one target probability per class for each instance (such as **one-hot vectors**, e.g., `[0., 0., 0., 1., 0., 0., 0., 0., 0., 0.]` to represent class 3), then we would need to use the `"categorical_crossentropy"` loss instead.
- If we were doing **binary classification** or multilabel binary classification, then we would use the `"sigmoid"` activation function in the output layer instead of the `"softmax"` activation function, and we would use the `"binary_crossentropy"` loss.

NOTE: Class labels

- If you want to convert sparse labels (i.e., class indices) to one-hot vector labels, use the `tf.keras.utils.to_categorical()` function.
- To go the other way round, use the `np.argmax()` function with `axis=1`.

Optimizer

- Regarding the optimizer, `"sgd"` means that we will train the model using stochastic gradient descent.
- In other words, Keras will perform the backpropagation algorithm described earlier (i.e., reverse-mode autodiff plus gradient descent).
- When using the `SGD` optimizer, it is important to tune the learning rate.
- So, you will generally want to use

```
optimizer=tf.keras.optimizers.SGD(learning_rate=__???__)
```

to set the learning rate, rather than `optimizer="sgd"`, which defaults to a learning rate of 0.01.

Training and evaluating the model

- To train the model call its `fit()` method:

```
>>> history = model.fit(X_train, y_train, epochs=30,
...                     validation_data=(X_valid, y_valid))
...
Epoch 1/30
1719/1719 [==============================] - 2s 989us/step
  - loss: 0.7220 - sparse_categorical_accuracy: 0.7649
  - val_loss: 0.4959 - val_sparse_categorical_accuracy: 0.8332
Epoch 2/30
1719/1719 [==============================] - 2s 964us/step
  - loss: 0.4825 - sparse_categorical_accuracy: 0.8332
  - val_loss: 0.4567 - val_sparse_categorical_accuracy: 0.8384
[...]
Epoch 30/30
1719/1719 [==============================] - 2s 963us/step
  - loss: 0.2235 - sparse_categorical_accuracy: 0.9200
  - val_loss: 0.3056 - val_sparse_categorical_accuracy: 0.8894
```

Training and evaluating the model

- We pass it the input features (`X_train`) and the target classes (`y_train`), as well as the number of epochs to train (or else it would default to just 1, which would definitely not be enough to converge to a good solution).
- We also pass a validation set (this is optional).
- Keras will measure the loss and the extra metrics on this set at the end of each epoch, which is very useful to see how well the model really performs.
- If the performance on the training set is much better than on the validation set, your model is probably overfitting the training set, or there is a bug, such as a data mismatch between the training set and the validation set.

Training and evaluating the model: Explanations

- The neural network is trained. At each epoch during training, Keras displays the number of mini-batches processed so far on the left side of the progress bar.
- The batch size is 32 by default, and since the training set has 55,000 images, the model goes through 1,719 batches per epoch: 1,718 of size 32, and 1 of size 24.
- After the progress bar, you can see the mean training time per sample, and the loss and accuracy (or any other extra metrics you asked for) on both the training set and the validation set.
- Notice that the training loss went down, which is a good sign, and the validation accuracy reached 88.94% after 30 epochs.
- That's slightly below the training accuracy, so there is a little bit of overfitting going on, but not a huge amount.

NOTE

- Instead of passing a validation set using the `validation_data` argument, you could set `validation_split` to the ratio of the training set that you want Keras to use for validation.
- For example, `validation_split=0.1` tells Keras to use the last 10% of the data (before shuffling) for validation.

Skewed Training Data

- If the training set was very skewed, with some classes being overrepresented and others underrepresented, it would be useful to set the `class_weight` argument when calling the `fit()` method, to **give a larger weight to underrepresented classes and a lower weight to overrepresented classes.**
- These weights would be used by Keras when computing the loss.
- If you need per-instance weights, set the `sample_weight` argument. If both `class_weight` and `sample_weight` are provided, then Keras multiplies them.
- Per-instance weights could be useful, for example, if some instances were labeled by experts while others were labeled using a crowdsourcing platform: you might want to give more weight to the former.

Plotting the Results

- The `fit()` method returns a `History` object containing the training parameters (`history.params`), the list of epochs it went through (`history.epoch`), and most importantly a dictionary (`history.history`) containing the loss and extra metrics it measured at the end of each epoch on the training set and on the validation set (if any).
- If you use this dictionary to create a Pandas DataFrame and call its `plot()` method, you get the learning curves shown in Figure 10-11:

Plotting the Results

- Both the training accuracy and the validation accuracy steadily increase during training, while the training loss and the validation loss decrease. This is good.
- The validation curves are relatively close to each other at first, but they get further apart over time, which shows that there's a little bit of overfitting.
- In this particular case, the model looks like it performed better on the validation set than on the training set at the beginning of training, but that's not actually the case:
  - The validation error is computed at the end of each epoch, while the training error is computed using a running mean during each epoch, so the training curve should be shifted by half an epoch to the left. If you do that, you will see that the training and validation curves overlap almost perfectly at the beginning of training.

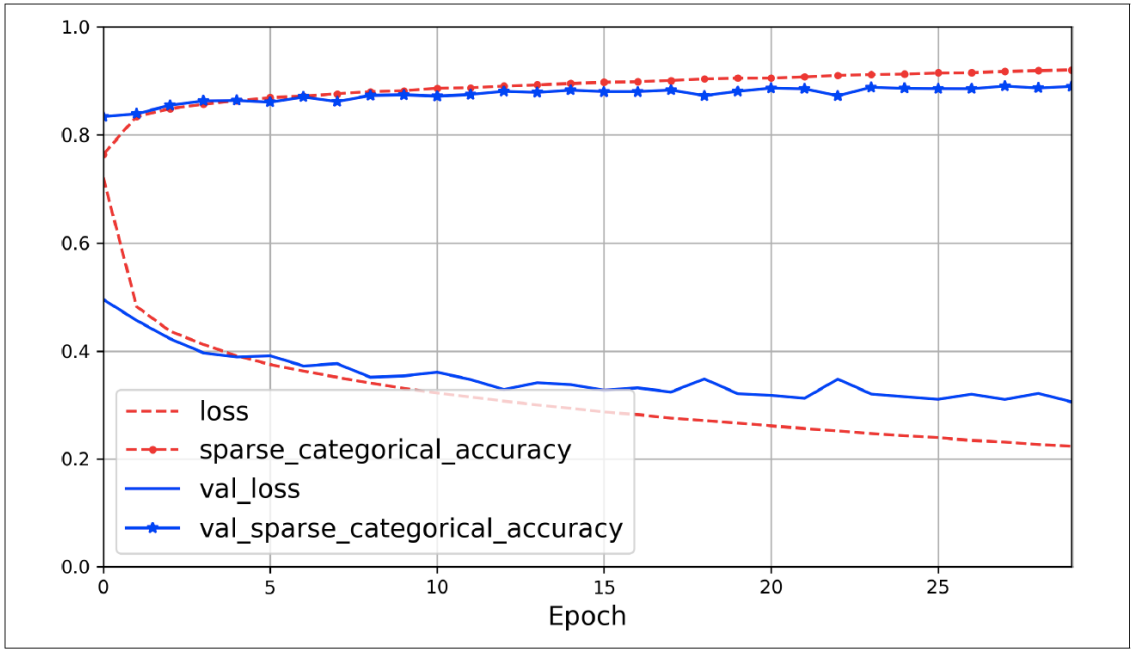


Figure 10-11. Learning curves: the mean training loss and accuracy measured over each epoch, and the mean validation loss and accuracy measured at the end of each epoch

Improving The Results

- The training set performance ends up beating the validation performance, as is generally the case when you train for long enough.
- You can tell that the model has not quite converged yet, as the validation loss is still going down, so you should probably continue training.
- This is as simple as calling the `fit()` method again, since Keras just continues training where it left off: you should be able to reach about 89.8% validation accuracy, while the training accuracy will continue to rise up to 100% (this is not always the case).

Improving The Results

- If you are not satisfied with the performance of your model, you should go back and tune the hyperparameters.
- The first one to check is the learning rate.
- If that doesn't help, try another optimizer (and always retune the learning rate after changing any hyperparameter).
- If the performance is still not great, then try tuning model hyperparameters such as the number of layers, the number of neurons per layer, and the types of activation functions to use for each hidden layer.
- You can also try tuning other hyperparameters, such as the batch size (it can be set in the `fit()` method using the `batch_size` argument, which defaults to 32).

Testing the model

- Once you are satisfied with your model's validation accuracy, you should evaluate it on the test set to estimate the generalization error before you deploy the model to production.
- You can easily do this using the `evaluate()` method (it also supports several other arguments, such as `batch_size` and `sample_weight`; please check the documentation for more details):

```
>>> model.evaluate(X_test, y_test)
313/313 [==============================] - 0s 626us/step
  - loss: 0.3243 - sparse_categorical_accuracy: 0.8864
[0.3243169784545898, 0.8863999843597412]
```

- It is common to get slightly lower performance on the test set than on the validation set, because the hyperparameters are tuned on the validation set, not the test set (however, in this example, we did not do any hyperparameter tuning, so the lower accuracy is just bad luck).
- Remember to resist the temptation to tweak the hyperparameters on the test set, or else your estimate of the generalization error will be too optimistic.

Using the model to make predictions

- Now let's use the model's `predict()` method to make predictions on new instances.
- Since we don't have actual new instances, we'll just use the first three instances of the test set:

```
>>> X_new = X_test[:3]
>>> y_proba = model.predict(X_new)
>>> y_proba.round(2)
array([[0.  , 0.  , 0.  , 0.  , 0.  , 0.01, 0.  , 0.02, 0.  , 0.97],
       [0.  , 0.  , 0.99, 0.  , 0.01, 0.  , 0.  , 0.  , 0.  , 0.  ],
       [0.  , 1.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  ]],
      dtype=float32)
```

- For each instance the model estimates one probability per class, from class 0 to class 9. This is similar to the output of the `predict_proba()` method in Scikit-Learn classifiers.

Using the model to make predictions

- For example, for the first image it estimates that the probability of class 9 (ankle boot) is 96%, the probability of class 7 (sneaker) is 2%, the probability of class 5 (sandal) is 1%, and the probabilities of the other classes are negligible.
- In other words, it is highly confident that the first image is footwear, most likely ankle boots but possibly sneakers or sandals.
- If you only care about the class with the highest estimated probability (even if that probability is quite low), then you can use the `argmax()` method to get the highest probability class index for each instance:

```
>>> import numpy as np
>>> y_pred = y_proba.argmax(axis=-1)
>>> y_pred
array([9, 2, 1])
>>> np.array(class_names)[y_pred]
array(['Ankle boot', 'Pullover', 'Trouser'], dtype='<U11')
```

Using the model to make predictions

- Here, the classifier actually classified all three images correctly (these images are shown in Figure 10-12):

```
>>> y_new = y_test[:3]
>>> y_new
array([9, 2, 1], dtype=uint8)
```

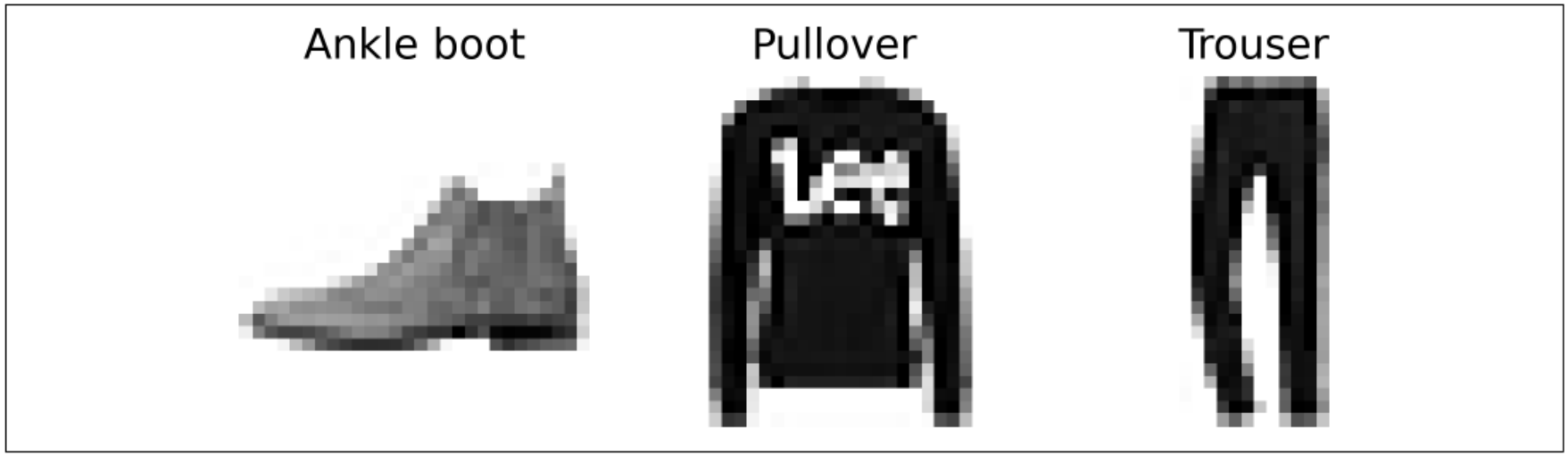


Figure 10-12. Correctly classified Fashion MNIST images

Building a Regression MLP Using the Sequential API

Building a Regression MLP Using the Sequential API

- California housing problem and tackle it using a MLP, with 3 hidden layers composed of 50 neurons each, but this time building it with Keras.
- Using the sequential API to build, train, evaluate, and use a regression MLP is quite similar to what we did for classification.
- The **main differences** in the following code example are the fact that:
  - the output layer has a single neuron (since we only want to predict a single value)
  - it uses no activation function,
  - the loss function is the mean squared error,
  - the metric is the RMSE,
  - we're using an Adam optimizer like Scikit-Learn's **`MLPRegressor`**
- In this example we don't need a `Flatten` layer, and instead we're using a `Normalization` layer as the first layer:
  - it does the same thing as Scikit-Learn's `StandardScaler`, but it must be fitted to the training data using its `adapt()` method before you call the model's `fit()` method.

Building a Regression MLP Using the Sequential API

```
tf.random.set_seed(42)
norm_layer = tf.keras.layers.Normalization(input_shape=X_train.shape[1:])
model = tf.keras.Sequential([
    norm_layer,
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(1)
])
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss="mse", optimizer=optimizer, metrics=["RootMeanSquaredError"])
norm_layer.adapt(X_train)
history = model.fit(X_train, y_train, epochs=20,
                    validation_data=(X_valid, y_valid))
mse_test, rmse_test = model.evaluate(X_test, y_test)
X_new = X_test[:3]
y_pred = model.predict(X_new)
```

NOTE

- The `Normalization` layer learns the feature means and standard deviations in the training data when you call the `adapt()` method.
- Yet when you display the model's summary, these statistics are listed as non-trainable. This is because these parameters are not affected by gradient descent.

Building Complex Models Using the Functional API

Building Complex Models Using the Functional API

- The sequential API is quite clean and straightforward.
- However, although `Sequential` models are extremely common, it is sometimes useful to build neural networks with more complex topologies, or with multiple inputs or outputs.
- For this purpose, Keras offers the functional API.
- One example of a nonsequential neural network is a **Wide & Deep neural network**. This neural network architecture was introduced in a 2016 paper.
  - It connects all or part of the inputs directly to the output layer, as shown in Figure 10-13.
  - This architecture makes it possible for the neural network to learn both deep patterns (using the deep path) and simple rules (through the short path). The short path can also be used to provide **manually engineered** features to the neural network.
- In contrast, a regular MLP forces all the data to flow through the full stack of layers; thus, simple patterns in the data may end up being distorted by this sequence of transformations.

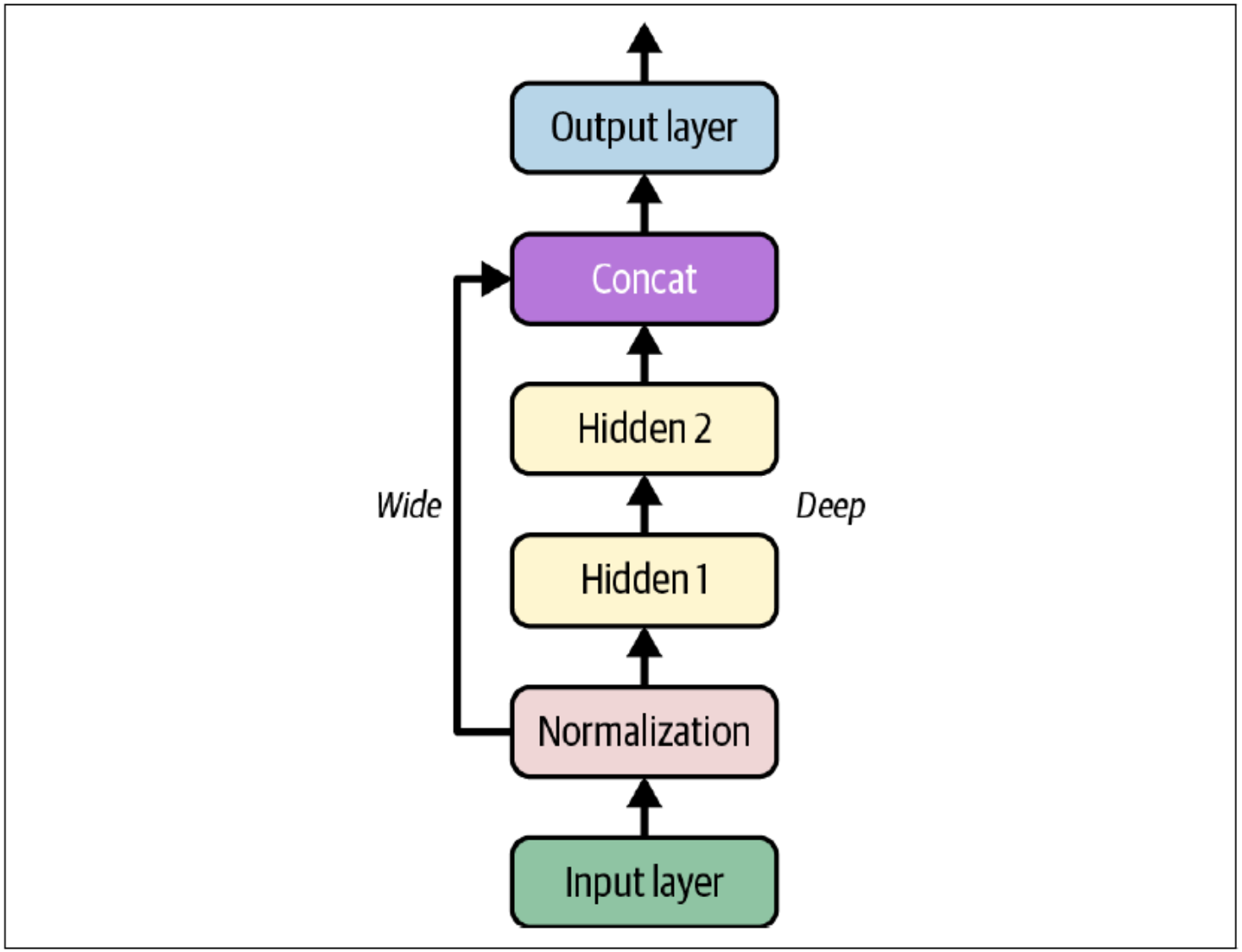


Figure 10-13. Wide & Deep neural network

Build Wide & Deep network

- At a high level,
- the **first five lines** create all the layers we need to build the model,
- the **next six lines** use these layers just like functions to go from the input to the output,
- the last line creates a Keras `Model` object by pointing to the input and the output.

```
normalization_layer = tf.keras.layers.Normalization()
hidden_layer1 = tf.keras.layers.Dense(30, activation="relu")
hidden_layer2 = tf.keras.layers.Dense(30, activation="relu")
concat_layer = tf.keras.layers.Concatenate()
output_layer = tf.keras.layers.Dense(1)

input_ = tf.keras.layers.Input(shape=X_train.shape[1:])
normalized = normalization_layer(input_)
hidden1 = hidden_layer1(normalized)
hidden2 = hidden_layer2(hidden1)
concat = concat_layer([normalized, hidden2])
output = output_layer(concat)

model = tf.keras.Model(inputs=[input_], outputs=[output])
```

Build Wide & Deep network: Details

- **First, we create five layers:**
- a `Normalization` layer to standardize the inputs,
- two `Dense` layers with 30 neurons each, using the ReLU activation function,
- a `Concatenate` layer, and
- one more `Dense` layer with a single neuron for the output layer, without any activation function.

```
normalization_layer = tf.keras.layers.Normalization()
hidden_layer1 = tf.keras.layers.Dense(30, activation="relu")
hidden_layer2 = tf.keras.layers.Dense(30, activation="relu")
concat_layer = tf.keras.layers.Concatenate()
output_layer = tf.keras.layers.Dense(1)
```

```
input_ = tf.keras.layers.Input(shape=X_train.shape[1:])
normalized = normalization_layer(input_)
hidden1 = hidden_layer1(normalized)
hidden2 = hidden_layer2(hidden1)
concat = concat_layer([normalized, hidden2])
output = output_layer(concat)

model = tf.keras.Model(inputs=[input_], outputs=[output])
```

Build Wide & Deep network: Details

- **Next**, we create an `Input` object (the variable name `input_` is used to avoid overshadowing Python's built-in `input()` function).
- This is a specification of the kind of input the model will get, including its `shape` and optionally its `dtype`, which defaults to 32-bit floats.
- A model may actually have multiple inputs, as you will see shortly.

```
normalization_layer = tf.keras.layers.Normalization()
hidden_layer1 = tf.keras.layers.Dense(30, activation="relu")
hidden_layer2 = tf.keras.layers.Dense(30, activation="relu")
concat_layer = tf.keras.layers.Concatenate()
output_layer = tf.keras.layers.Dense(1)

input_ = tf.keras.layers.Input(shape=X_train.shape[1:])
normalized = normalization_layer(input_)
hidden1 = hidden_layer1(normalized)
hidden2 = hidden_layer2(hidden1)
concat = concat_layer([normalized, hidden2])
output = output_layer(concat)

model = tf.keras.Model(inputs=[input_], outputs=[output])
```

Build Wide & Deep network: Details

- Then we use the `Normalization` layer just like a function, passing it the `Input` object.
- This is why this is called the functional API.
- Note that we are just telling Keras how it should connect the layers together; no actual data is being processed yet, as the `Input` object is just a data specification.
- In other words, it's a symbolic input.
- The output of this call is also symbolic: `normalized` doesn't store any actual data, it's just used to construct the model.

```
normalization_layer = tf.keras.layers.Normalization()
hidden_layer1 = tf.keras.layers.Dense(30, activation="relu")
hidden_layer2 = tf.keras.layers.Dense(30, activation="relu")
concat_layer = tf.keras.layers.Concatenate()
output_layer = tf.keras.layers.Dense(1)

input_ = tf.keras.layers.Input(shape=X_train.shape[1:])
normalized = normalization_layer(input_)
hidden1 = hidden_layer1(normalized)
hidden2 = hidden_layer2(hidden1)
concat = concat_layer([normalized, hidden2])
output = output_layer(concat)

model = tf.keras.Model(inputs=[input_], outputs=[output])
```

Build Wide & Deep network: Details

- In the same way, we then pass `normalized` to `hidden_layer1`, which outputs `hidden1`, and we pass `hidden1` to `hidden_layer2`, which outputs `hidden2`.

```
normalization_layer = tf.keras.layers.Normalization()
hidden_layer1 = tf.keras.layers.Dense(30, activation="relu")
hidden_layer2 = tf.keras.layers.Dense(30, activation="relu")
concat_layer = tf.keras.layers.Concatenate()
output_layer = tf.keras.layers.Dense(1)

input_ = tf.keras.layers.Input(shape=X_train.shape[1:])
normalized = normalization_layer(input_)
hidden1 = hidden_layer1(normalized)
hidden2 = hidden_layer2(hidden1)
concat = concat_layer([normalized, hidden2])
output = output_layer(concat)

model = tf.keras.Model(inputs=[input_], outputs=[output])
```

Build Wide & Deep network: Details

- So far we've connected the layers sequentially, but then we use the `concat_layer` to concatenate the input and the second hidden layer's output.
- Again, no actual data is concatenated yet: it's all **symbolic**, to build the model.

```
normalization_layer = tf.keras.layers.Normalization()
hidden_layer1 = tf.keras.layers.Dense(30, activation="relu")
hidden_layer2 = tf.keras.layers.Dense(30, activation="relu")
concat_layer = tf.keras.layers.Concatenate()
output_layer = tf.keras.layers.Dense(1)

input_ = tf.keras.layers.Input(shape=X_train.shape[1:])
normalized = normalization_layer(input_)
hidden1 = hidden_layer1(normalized)
hidden2 = hidden_layer2(hidden1)
concat = concat_layer([normalized, hidden2])
output = output_layer(concat)

model = tf.keras.Model(inputs=[input_], outputs=[output])
```

Build Wide & Deep network: Details

- Then we pass `concat` to the `output_layer`, which gives us the final `output`.
- Lastly, we create a Keras `Model`, specifying which inputs and outputs to use.
- Once you have built this Keras model, everything is exactly like earlier, so there's no need to repeat it here: you compile the model, adapt the `Normalization` layer, fit the model, evaluate it, and use it to make predictions.

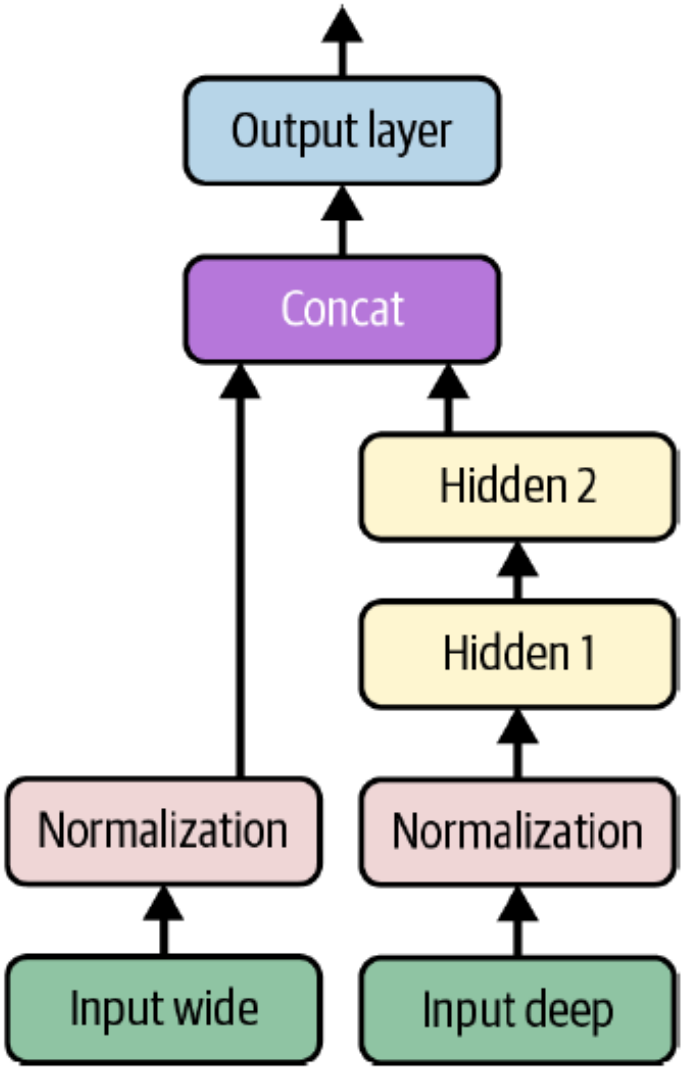
```
normalization_layer = tf.keras.layers.Normalization()
hidden_layer1 = tf.keras.layers.Dense(30, activation="relu")
hidden_layer2 = tf.keras.layers.Dense(30, activation="relu")
concat_layer = tf.keras.layers.Concatenate()
output_layer = tf.keras.layers.Dense(1)

input_ = tf.keras.layers.Input(shape=X_train.shape[1:])
normalized = normalization_layer(input_)
hidden1 = hidden_layer1(normalized)
hidden2 = hidden_layer2(hidden1)
concat = concat_layer([normalized, hidden2])
output = output_layer(concat)

model = tf.keras.Model(inputs=[input_], outputs=[output])
```

Build Wide & Deep network – Multiple Inputs

- But what if you want to send a subset of the features through the wide path and a different subset (possibly overlapping) through the deep path, as illustrated in Figure 10-14? In this case, one solution is to use multiple inputs.
- For example, suppose we want to send five features through the wide path (features 0 to 4), and six features through the deep path (features 2 to 7). We can do this as follows:



Build Wide & Deep network – Multiple Inputs

- A few things to note in this example, compared to the previous one:
- Each `Dense` layer is created and called on the same line. This is a common practice, as it makes the code more concise without losing clarity.
- However, we can't do this with the `Normalization` layer since we need a reference to the layer to be able to call its `adapt()` method before fitting the model.

```
input_wide = tf.keras.layers.Input(shape=[5])  # features 0 to 4
input_deep = tf.keras.layers.Input(shape=[6])  # features 2 to 7
norm_layer_wide = tf.keras.layers.Normalization()
norm_layer_deep = tf.keras.layers.Normalization()
norm_wide = norm_layer_wide(input_wide)
norm_deep = norm_layer_deep(input_deep)
hidden1 = tf.keras.layers.Dense(30, activation="relu")(norm_deep)
hidden2 = tf.keras.layers.Dense(30, activation="relu")(hidden1)
concat = tf.keras.layers.concatenate([norm_wide, hidden2])
output = tf.keras.layers.Dense(1)(concat)
model = tf.keras.Model(inputs=[input_wide, input_deep], outputs=[output])
```

Build Wide & Deep network – Multiple Inputs

- We used `tf.keras.layers.concatenate()`, which creates a `Concatenate` layer and calls it with the given inputs.
- We specified `inputs=[input_wide, input_deep]` when creating the model, since there are two inputs.

```
input_wide = tf.keras.layers.Input(shape=[5])  # features 0 to 4
input_deep = tf.keras.layers.Input(shape=[6])  # features 2 to 7
norm_layer_wide = tf.keras.layers.Normalization()
norm_layer_deep = tf.keras.layers.Normalization()
norm_wide = norm_layer_wide(input_wide)
norm_deep = norm_layer_deep(input_deep)
hidden1 = tf.keras.layers.Dense(30, activation="relu")(norm_deep)
hidden2 = tf.keras.layers.Dense(30, activation="relu")(hidden1)
concat = tf.keras.layers.concatenate([norm_wide, hidden2])
output = tf.keras.layers.Dense(1)(concat)
model = tf.keras.Model(inputs=[input_wide, input_deep], outputs=[output])
```

Build Wide & Deep network – Multiple Inputs

- Now we can compile the model as usual, but when we call the `fit()` method, instead of passing a single input matrix `X_train`, we must pass a pair of matrices `(X_train_wide, X_train_deep)`, one per input.
- The same is true for `X_valid`, and also for `X_test` and `X_new` when you call `evaluate()` or `predict()`:

```
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss="mse", optimizer=optimizer, metrics=["RootMeanSquaredError"])

X_train_wide, X_train_deep = X_train[:, :5], X_train[:, 2:]
X_valid_wide, X_valid_deep = X_valid[:, :5], X_valid[:, 2:]
X_test_wide, X_test_deep = X_test[:, :5], X_test[:, 2:]
X_new_wide, X_new_deep = X_test_wide[:3], X_test_deep[:3]

norm_layer_wide.adapt(X_train_wide)
norm_layer_deep.adapt(X_train_deep)
history = model.fit((X_train_wide, X_train_deep), y_train, epochs=20,
                    validation_data=((X_valid_wide, X_valid_deep), y_valid))
mse_test = model.evaluate((X_test_wide, X_test_deep), y_test)
y_pred = model.predict((X_new_wide, X_new_deep))
```

Build Wide & Deep network – Multiple Inputs

Instead of passing a tuple `(X_train_wide, X_train_deep)`, you can pass a dictionary `{"input_wide": X_train_wide, "input_deep": X_train_deep}`, if you set `name="input_wide"` and `name="input_deep"` when creating the inputs. This is highly recommended when there are many inputs, to clarify the code and avoid getting the order wrong.

Build Wide & Deep network – Multiple Outputs

- There are also many use cases in which you may want to have multiple outputs:
  - The task may demand it. For instance, you may want to locate and classify the main object in a picture. This is both a regression tasks and a classification task.
  - You may have multiple independent tasks based on the same data.
  - You could train one neural network per task, but in many cases you will get better results on all tasks by training a single neural network with one output per task.
  - This is because the neural network can learn features in the data that are useful across tasks.
  - For example, you could perform multitask classification on pictures of faces, using one output to classify the person's facial expression (smiling, surprised, etc.) and another output to identify whether they are wearing glasses or not.

Build Wide & Deep network – Multiple Outputs

- Another use case is as a regularization technique (i.e., a training constraint whose objective is to reduce overfitting and thus improve the model's ability to generalize).
- For example, you may want to add an auxiliary output in a neural network architecture (see Figure 10-15) to ensure that the underlying part of the network learns something useful on its own, without relying on the rest of the network.

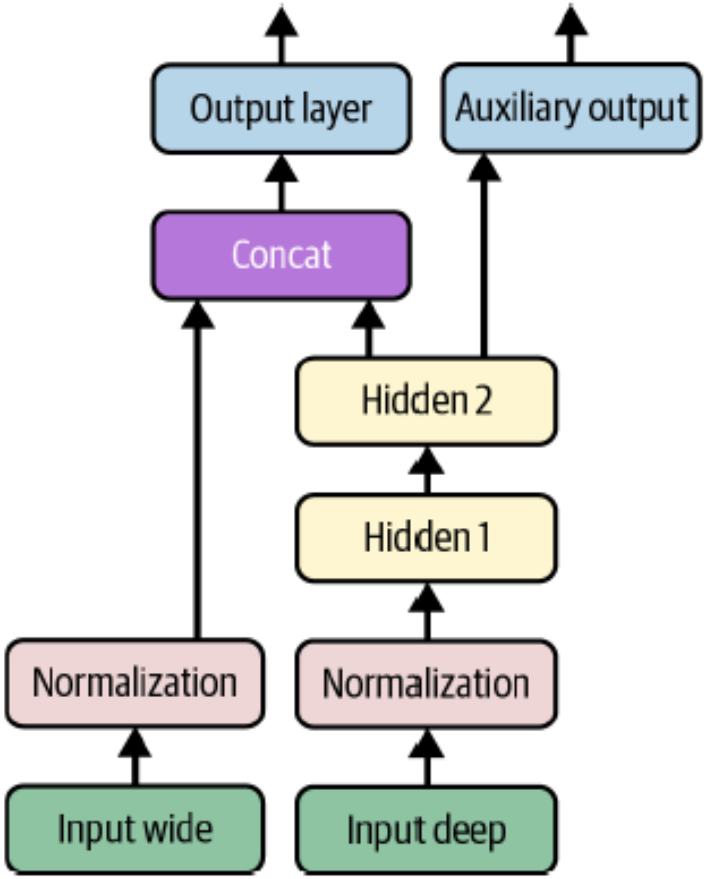


Figure 10-15. Handling multiple outputs, in this example to add an auxiliary output for regularization.

Build Wide & Deep network – Multiple Outputs

- Adding an extra output is quite easy: we just connect it to the appropriate layer and add it to the model's list of outputs.
- For example, the following code builds the network represented in Figure 10-15:

```
[...]  # Same as above, up to the main output layer
output = tf.keras.layers.Dense(1)(concat)
aux_output = tf.keras.layers.Dense(1)(hidden2)
model = tf.keras.Model(inputs=[input_wide, input_deep],
                       outputs=[output, aux_output])
```

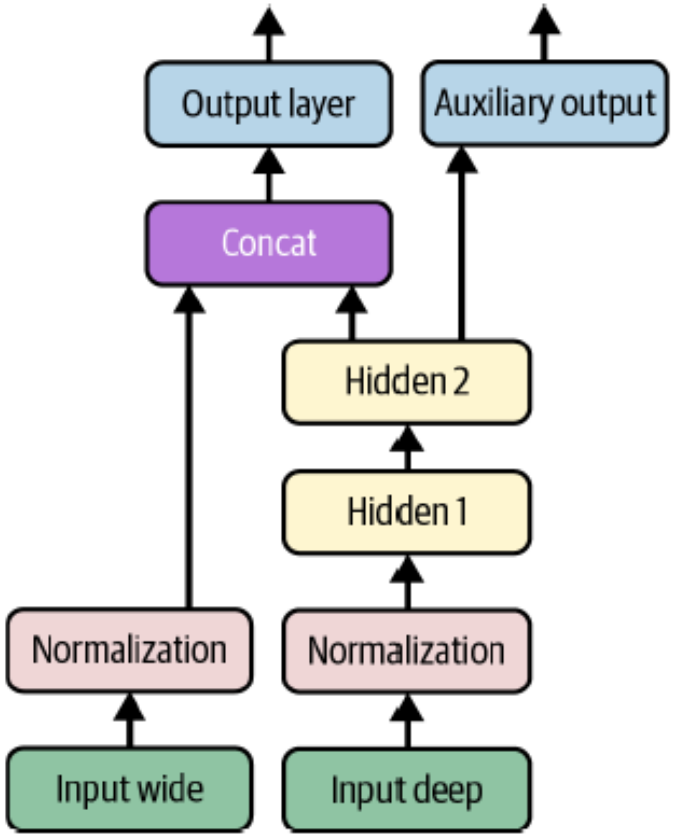


Figure 10-15. Handling multiple outputs, in this example to add an auxiliary output for regularization.

Build Wide & Deep network – Multiple Outputs

- Each output will need its own loss function. Therefore, when we compile the model, we should pass a list of losses.
- If we pass a single loss, Keras will assume that the same loss must be used for all outputs.
- By default, Keras will compute all the losses and simply add them up to get the final loss used for training.
- Since we care much more about the main output than about the auxiliary output (as it is just used for regularization), we want to give the main output's loss a much greater weight.
- Luckily, it is possible to set all the loss weights when compiling the model:

```
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss=("mse", "mse"), loss_weights=(0.9, 0.1), optimizer=optimizer,
              metrics=["RootMeanSquaredError"])
```

- Instead of passing a tuple `loss=("mse", "mse")`, you can pass a dictionary `loss={"output": "mse", "aux_output": "mse"}`, assuming you created the output layers with `name="output"` and `name="aux_output"`.
- Just like for the inputs, this clarifies the code and avoids errors when there are several outputs. You can also pass a dictionary for `loss_weights`.

Build Wide & Deep network – Multiple Outputs

- Now when we train the model, we need to provide labels for each output.
- In this example, the main output and the auxiliary output should try to predict the same thing, so they should use the same labels.
- So instead of passing `y_train`, we need to pass `(y_train, y_train)`, or a dictionary `{"output": y_train, "aux_output":y_train}` if the outputs were named `"output"` and `"aux_output"`.
- The same goes for `y_valid` and `y_test`:

```
norm_layer_wide.adapt(X_train_wide)
norm_layer_deep.adapt(X_train_deep)
history = model.fit(
    (X_train_wide, X_train_deep), (y_train, y_train), epochs=20,
    validation_data=((X_valid_wide, X_valid_deep), (y_valid, y_valid))
)
```

Build Wide & Deep network – Multiple Outputs

- When we evaluate the model, Keras returns the weighted sum of the losses, as well as all the individual losses and metrics:

```
eval_results = model.evaluate((X_test_wide, X_test_deep), (y_test, y_test))
weighted_sum_of_losses, main_loss, aux_loss, main_rmse, aux_rmse = eval_results
```

  - If you set `return_dict=True`, then `evaluate()` will return a dictionary instead of a big tuple.
- Similarly, the `predict()` method will return predictions for each output:

```
y_pred_main, y_pred_aux = model.predict((X_new_wide, X_new_deep))
```

- The `predict()` method returns a tuple, and it does not have a `return_dict` argument to get a dictionary instead. However, you can

```
y_pred_tuple = model.predict((X_new_wide, X_new_deep))
y_pred = dict(zip(model.output_names, y_pred_tuple))
```

Using the Subclassing API to Build Dynamic Models

Sequential API and the Functional API

- Both the sequential API and the functional API are declarative: you start by declaring which layers you want to use and how they should be connected, and only then can you start feeding the model some data for training or inference.
- This has many advantages: the model can easily be saved, cloned, and shared; its structure can be displayed and analyzed; the framework can infer shapes and check types, so errors can be caught early (i.e., before any data ever goes through the model).
- It's also fairly straightforward to debug, since the whole model is a static graph of layers.
- But the flip side is just that: it's static. Some models involve loops, varying shapes, conditional branching, and other dynamic behaviors.
- For such cases, or simply if you prefer a more imperative programming style, the subclassing API is useful.

Subclassing API

- With the Subclassing API, you subclass the `Model` class, create the layers you need in the constructor, and use them to perform the forward pass computations you want in the `call()` method.
- For example, creating an instance of the following `WideAndDeepModel` class gives us an equivalent model to the one we just built with the functional API:

```
class WideAndDeepModel(tf.keras.Model):
    def __init__(self, units=30, activation="relu", **kwargs):
        super().__init__(**kwargs)  # needed to support naming the model
        self.norm_layer_wide = tf.keras.layers.Normalization()
        self.norm_layer_deep = tf.keras.layers.Normalization()
        self.hidden1 = tf.keras.layers.Dense(units, activation=activation)
        self.hidden2 = tf.keras.layers.Dense(units, activation=activation)
        self.main_output = tf.keras.layers.Dense(1)
        self.aux_output = tf.keras.layers.Dense(1)

    def call(self, inputs):
        input_wide, input_deep = inputs
        norm_wide = self.norm_layer_wide(input_wide)
        norm_deep = self.norm_layer_deep(input_deep)
        hidden1 = self.hidden1(norm_deep)
        hidden2 = self.hidden2(hidden1)
        concat = tf.keras.layers.concatenate([norm_wide, hidden2])
        output = self.main_output(concat)
        aux_output = self.aux_output(hidden2)
        return output, aux_output

model = WideAndDeepModel(30, activation="relu", name="my_cool_model")
```

Subclassing API

- This example looks like the previous one, except we separate the creation of the layers in the constructor from their usage in the `call()` method.
- And we don't need to create the `Input` objects: we can use the `input` argument to the `call()` method.
- Now that we have a model instance, we can compile it, adapt its normalization layers (e.g., using `model.norm_layer_wide.adapt(...)` and `model.norm_layer_deep.adapt(...)`), fit it, evaluate it, and use it to make predictions, exactly like we did with the functional API.

Subclassing API

- The big difference with this API is that you can include pretty much anything you want in the `call()` method: `for` loops, `if` statements, low-level TensorFlow operations—your imagination is the limit (see Chapter 12)!
- This makes it a great API when experimenting with new ideas, especially for researchers.
- However, this extra flexibility does come at a cost:
  - your model's architecture is hidden within the `call()` method, so Keras cannot easily inspect it;
  - the model cannot be cloned using `tf.keras.models.clone_model()`;
  - and when you call the `summary()` method, you only get a list of layers, without any information on how they are connected to each other.
- Moreover, Keras cannot check types and shapes ahead of time, and it is easier to make mistakes.
- So unless you really need that extra flexibility, you should probably stick to the sequential API or the functional API.

Saving and Restoring a Model

Saving a Model

- To save a Keras model:

```
model.save("my_keras_model", save_format="tf")
```

- When you set `save_format="tf"`, Keras saves the model using TensorFlow's SavedModel format: this is a directory (with the given name) containing several files and subdirectories.
- The saved_model.pb file contains the model's architecture and logic in the form of a serialized computation graph, so you don't need to deploy the model's source code in order to use it in production; the SavedModel is sufficient (you will see how this works in Chapter 12).
- The keras_metadata.pb file contains extra information needed by Keras.
- The variables subdirectory contains all the parameter values (including the connection weights, the biases, the normalization statistics, and the optimizer's parameters), possibly split across multiple files if the model is very large.
- Lastly, the assets directory may contain extra files, such as data samples, feature names, class names, and so on. By default, the assets directory is empty. Since the optimizer is also saved, including its hyperparameters and any state it may have, after loading the model you can continue training if you want.

Saving a Model: NOTE

If you set `save_format="h5"` or use a filename that ends with *.h5*, *.hdf5*, or *.keras*, then Keras will save the model to a single file using a Keras-specific format based on the HDF5 format. However, most TensorFlow deployment tools require the SavedModel format instead.

Loading a Model

- You will typically have a script that trains a model and saves it, and one or more scripts (or web services) that load the model and use it to evaluate it or to make predictions. Loading the model is just as easy as saving it:

```
model = tf.keras.models.load_model("my_keras_model")
y_pred_main, y_pred_aux = model.predict((X_new_wide, X_new_deep))
```

- You can also use `save_weights()` and `load_weights()` to save and load only the parameter values. This includes the connection weights, biases, preprocessing stats, optimizer state, etc.
  - The parameter values are saved in one or more files such as my_weights.data-00004-of-00052, plus an index file like my_weights.index.
- Saving just the weights is faster and uses less disk space than saving the whole model, so it's perfect to save quick checkpoints during training. If you're training a big model, and it takes hours or days, then you must save checkpoints regularly in case the computer crashes.

Loading a Model

- Saving just the weights is faster and uses less disk space than saving the whole model, so it's perfect to save quick checkpoints during training. If you're training a big model, and it takes hours or days, then you must save checkpoints regularly in case the computer crashes.
- But how can you tell the `fit()` method to save checkpoints? Use callbacks.

Using Callbacks

- But how can you tell the `fit()` method to save checkpoints? Use callbacks.
- The `fit()` method accepts a `callbacks` argument that lets you specify a list of objects that Keras will call
  - before and after training,
  - before and after each epoch,
  - and even before and after processing each batch.
- For example, the `ModelCheckpoint` callback saves checkpoints of your model at regular intervals during training, by default at the end of each epoch:

```
checkpoint_cb = tf.keras.callbacks.ModelCheckpoint("my_checkpoints",
                                                   save_weights_only=True)
history = model.fit([...], callbacks=[checkpoint_cb])
```

Using Callbacks

- Moreover, if you use a validation set during training, you can set `save_best_only=True` when creating the `ModelCheckpoint`.
- In this case, it will only save your model when its performance on the validation set is the best so far.
- This way, you do not need to worry about training for too long and overfitting the training set: simply restore the last saved model after training, and this will be the best model on the validation set.
- This is one way to implement early stopping (introduced in Chapter 4), but it won't actually stop training.

Using Callbacks - EarlyStopping

- Another way is to use the `EarlyStopping` callback. It will interrupt training when it measures no progress on the validation set for a number of epochs (defined by the `patience` argument), and if you set `restore_best_weights=True` it will roll back to the best model at the end of training.
- You can combine both callbacks to save checkpoints of your model in case your computer crashes, and interrupt training early when there is no more progress, to avoid wasting time and resources and to reduce overfitting:

```
early_stopping_cb = tf.keras.callbacks.EarlyStopping(patience=10,
                                                     restore_best_weights=True)
history = model.fit([...], callbacks=[checkpoint_cb, early_stopping_cb])
```

- The number of epochs can be set to a large value since training will stop automatically when there is no more progress (just make sure the learning rate is not too small, or else it might keep making slow progress until the end).
- The `EarlyStopping` callback will store the weights of the best model in RAM, and it will restore them for you at the end of training.
- Many other callbacks are available in the `tf.keras.callbacks` package.

Using Callbacks - Custom

- If you need extra control, you can easily write your own custom callbacks.
- For example, the following custom callback will display the ratio between the validation loss and the training loss during training (e.g., to detect overfitting):

```
class PrintValTrainRatioCallback(tf.keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs):
        ratio = logs["val_loss"] / logs["loss"]
        print(f"Epoch={epoch}, val/train={ratio:.2f}")
```

- As you might expect, you can implement **`on_train_begin()`**, **`on_train_end()`**, **`on_epoch_begin()`**, **`on_epoch_end()`**, **`on_batch_begin()`**, and **`on_batch_end()`**.
- Callbacks can also be used during evaluation and predictions, should you ever need them (e.g., for debugging).
- For evaluation, you should implement `on_test_begin()`, `on_test_end()`, `on_test_batch_begin()`, or `on_test_batch_end()`, which are called by `evaluate()`.
- For prediction, you should implement `on_predict_begin()`, `on_predict_end()`, `on_predict_batch_begin()`, or `on_predict_batch_end()`, which are called by `predict()`.

Using TensorBoard for Visualization

- TensorBoard is a great interactive visualization tool that you can use to:
  - view the learning curves during training, compare curves and metrics between multiple runs,
  - visualize the computation graph,
  - analyze training statistics,
  - view images generated by your model,
  - visualize complex multidimensional data projected down to 3D and automatically clustered for you,
  - profile your network (i.e., measure its speed to identify bottlenecks), and more!

- For setup details see the book!

Using TensorBoard for Visualization

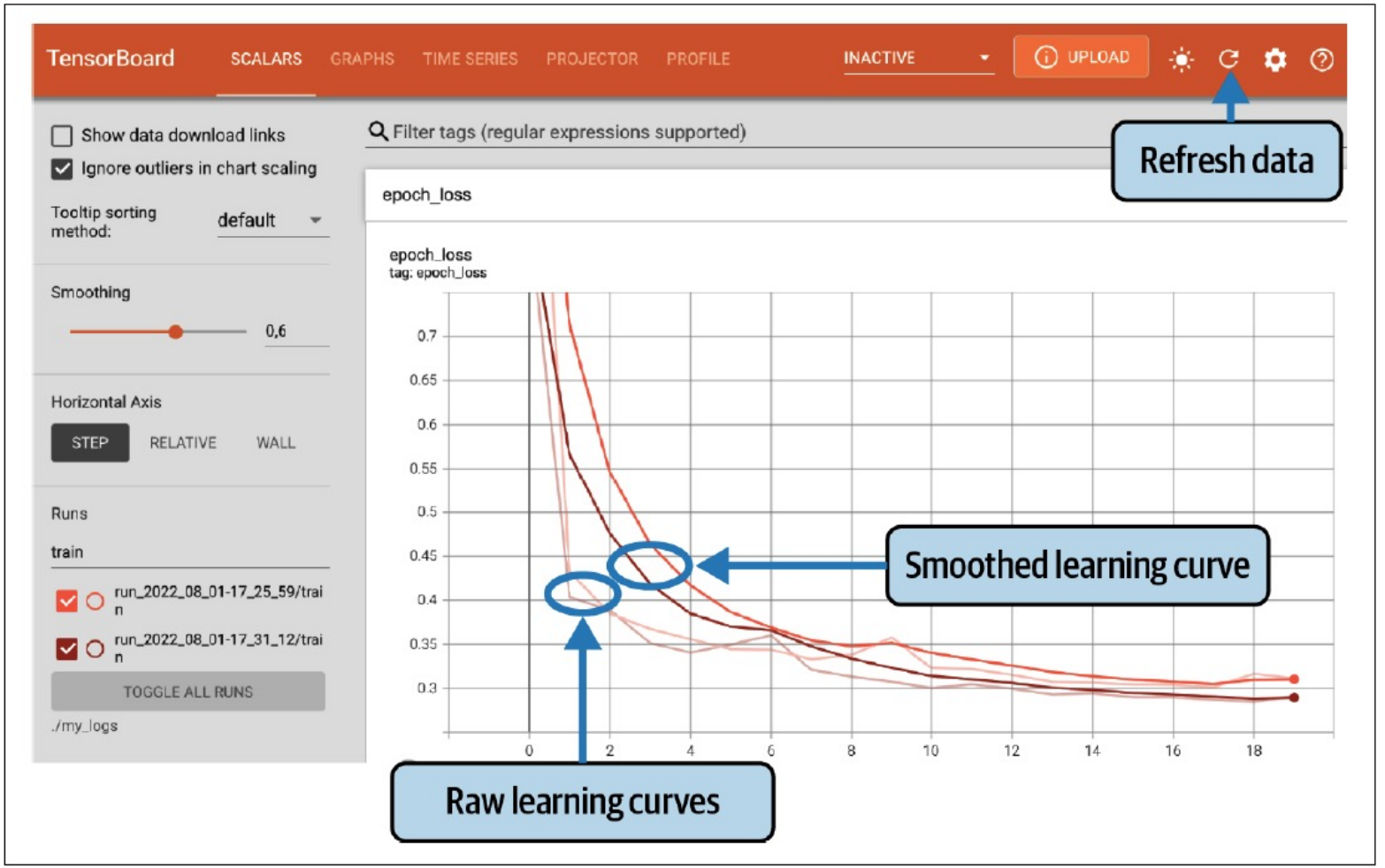


Figure 10-16. Visualizing learning curves with TensorBoard